

Unidad VI

Clase 10

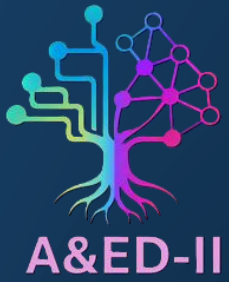
Árboles Avanzados

AVL Adelson-Velsky y Landis

ARBOL BINARIO DE BUSQUEDA

BALANCEADO

Entender por qué el balance es crítico en estructuras de búsqueda.



Actividad

Un Árbol Binario de Búsqueda puede respetar correctamente la propiedad de orden y, aun así, perder eficiencia si su forma queda desbalanceada.

Se considera que un nodo está desbalanceado si la diferencia absoluta entre la altura de su subárbol izquierdo y la altura de su subárbol derecho es mayor que 1.

```
Math.abs(altura(izquierdo) - altura(derecho)) > 1
```

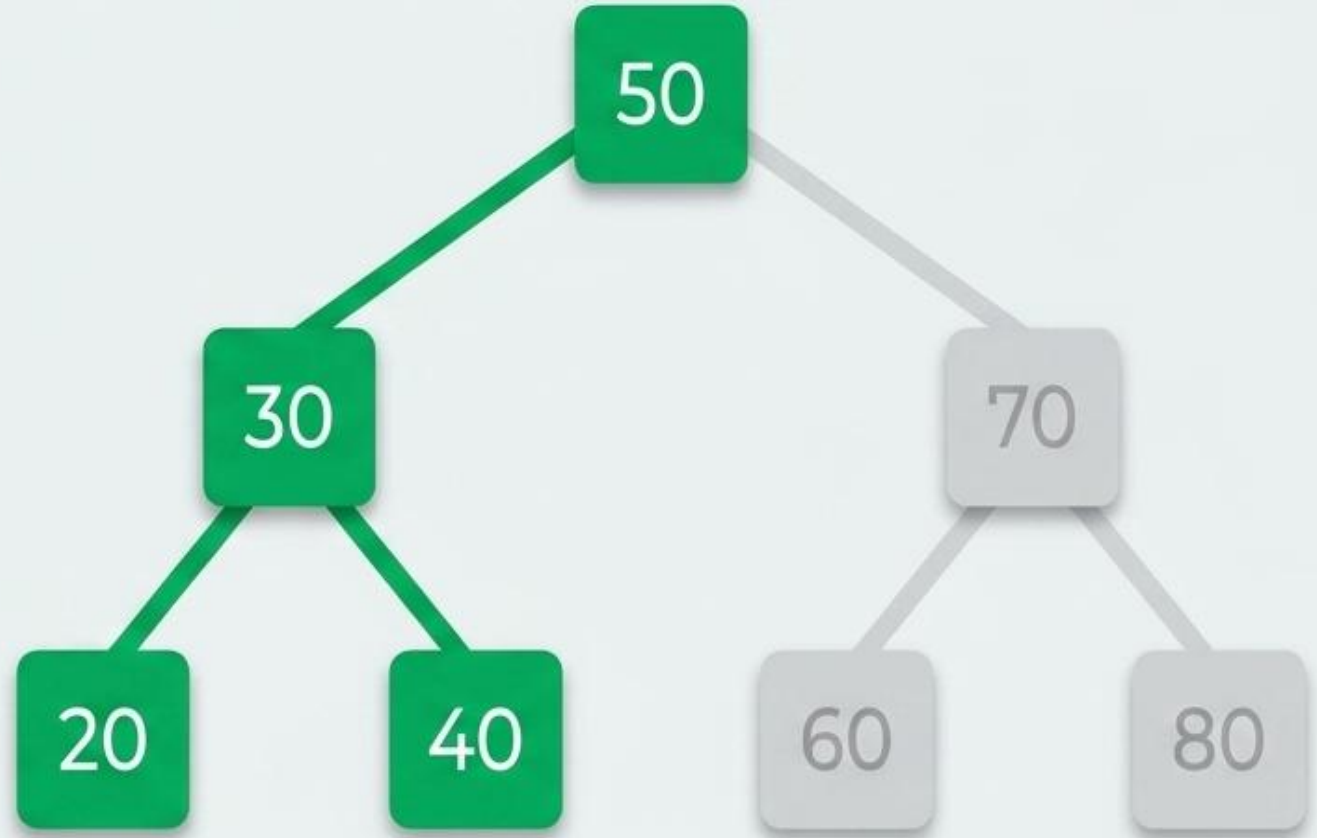
Dado un ABB de enteros, desarrollar una solución modular que permita listar los nodos desbalanceados.

El ABB como estructura de búsqueda ordenada

Árbol Binario de Búsqueda (ABB): Estructura jerárquica con una regla de orden estricta.

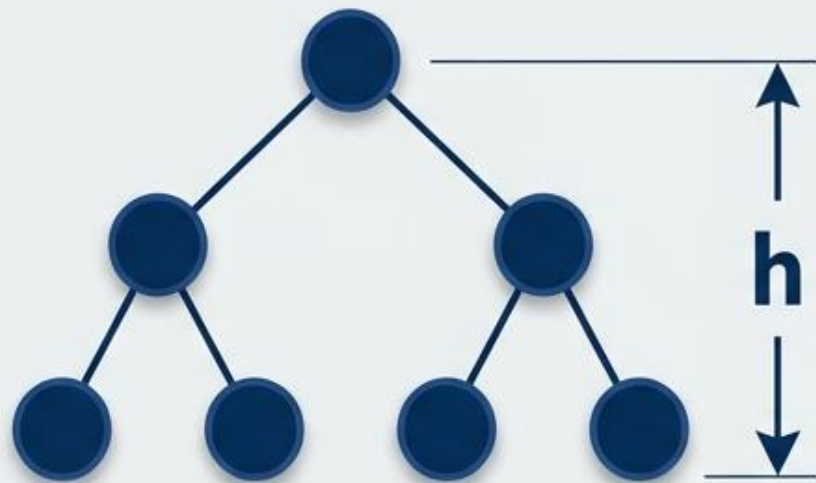
Subárbol Izquierdo: Contiene únicamente valores menores al nodo actual.

Subárbol Derecho: Contiene únicamente valores mayores al nodo actual.



El ABB no busca recorriendo todo: decide hacia dónde avanzar.

La eficiencia depende de la altura



Altura (h): Mayor distancia física desde la raíz hasta una hoja.

$$\text{Costo General} = O(h)$$

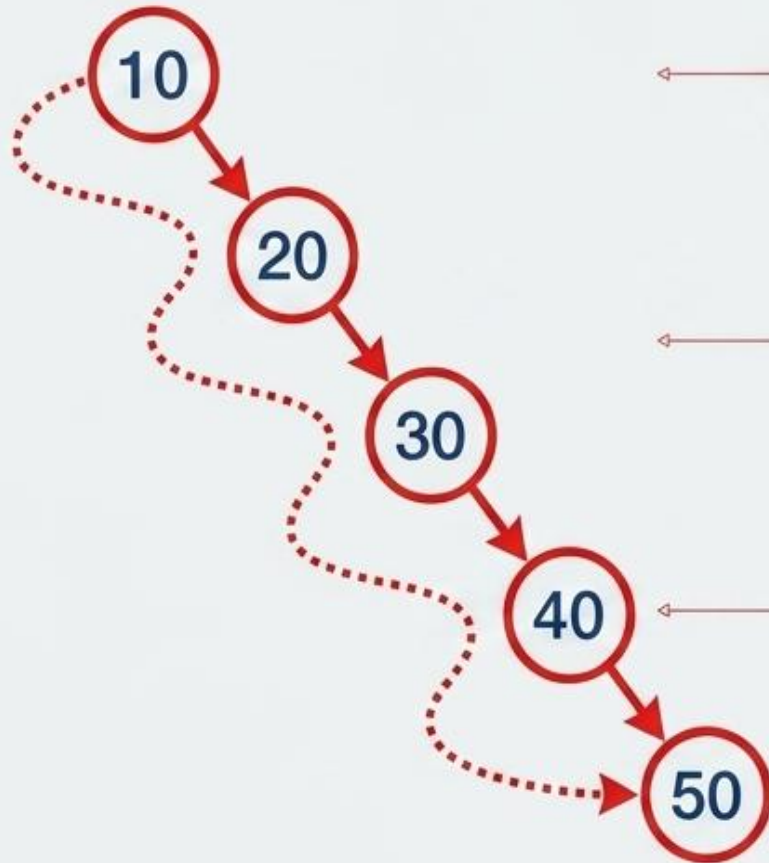
Costo Operativo:
Búsqueda, inserción y eliminación dependen de h.



La propiedad de orden permite buscar; la **altura** determina cuánto cuesta buscar.

Cuando el ABB se degrada

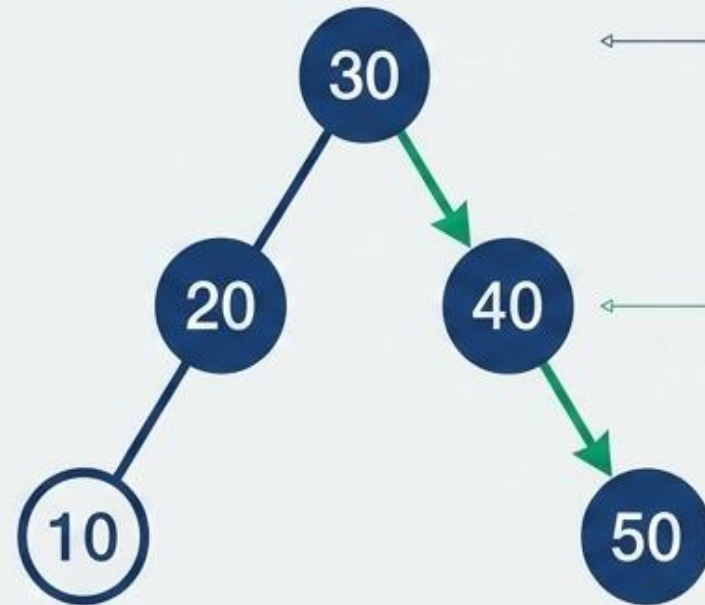
10 → 20 → 30 → 40 → 50



← Se ingresan datos en orden creciente.

← El árbol cumple la regla (mayores a la derecha), pero su forma física degenera en una lista.

← Buscar el valor 50 exige recorrer casi todos los nodos.



← Mismos datos, diferente distribución.

← Rutas de búsqueda cortas y eficientes.

Del costo $O(\log n)$ al costo $O(n)$

	ABB Balanceado 	ABB Degenerado 
Altura	Baja (cercana a $\log n$)	Alta (cercana a n)
Búsqueda	Eficiente / Logarítmica	Lenta / Lineal
Costo Aproximado	$O(\log n)$	$O(n)$

El problema no radica en la regla de ordenamiento,
sino en la falta de control sobre la forma física del árbol.

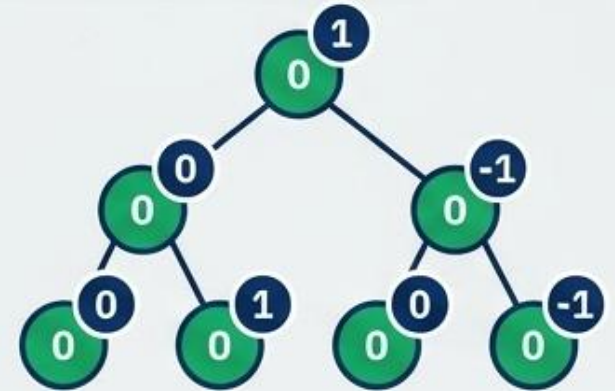
La necesidad de un árbol balanceado

¿Cómo mantener la ventaja del ABB aunque los datos ingresen en un orden desfavorable?



La Respuesta: Árbol AVL

- Mantiene intacta la propiedad de orden del ABB.
- Incorpora una condición de equilibrio: la altura de los subárboles no puede diferir en más de 1.
- Se **auto-regula** mediante operaciones de **rotación**.

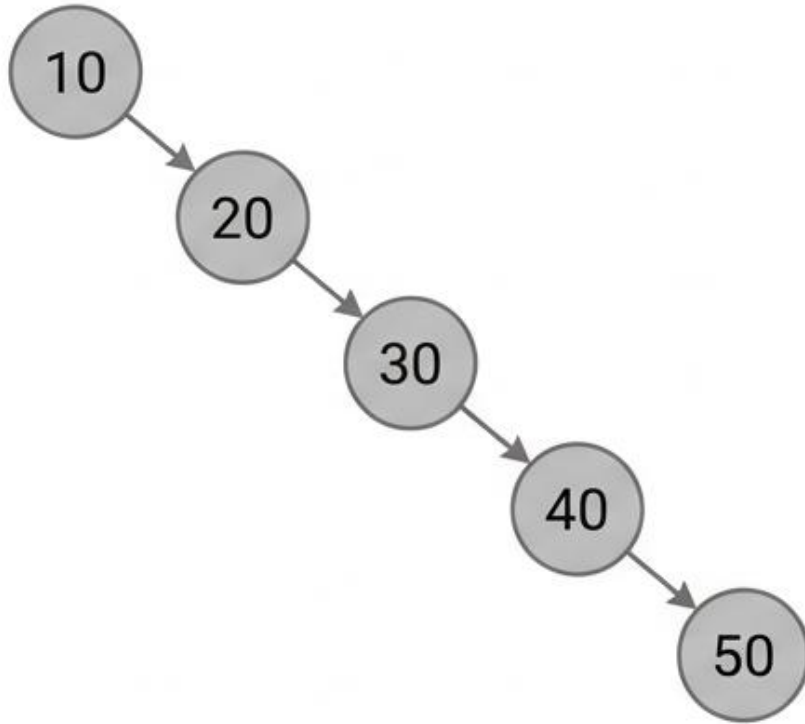


AVL no cambia la lógica de búsqueda: controla la forma del árbol para conservar eficiencia.

Árbol AVL: La evolución del ABB

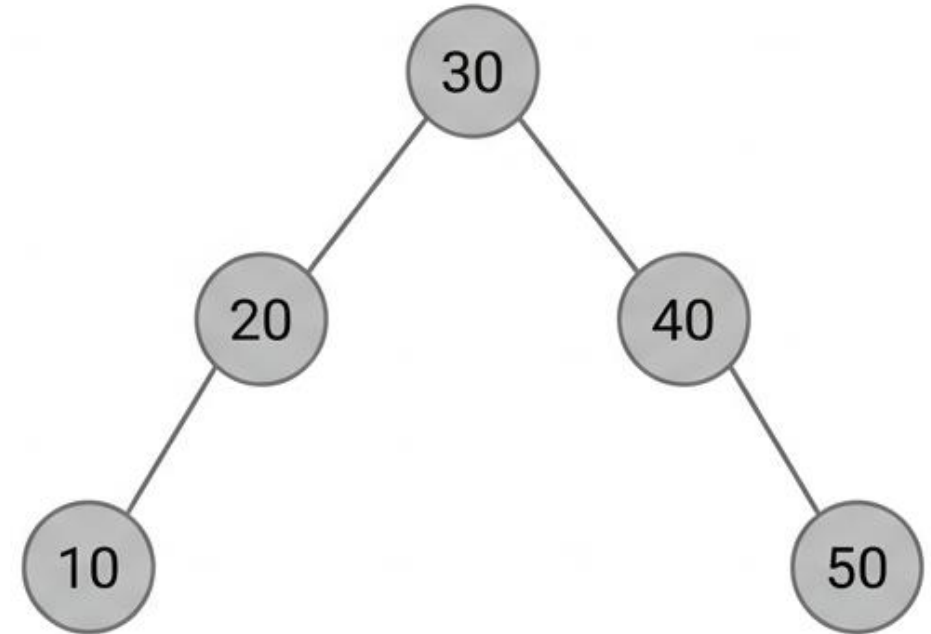
AVL = Creado por los matemáticos Adelson, Velskii y Landis.

ESTADO INICIAL: ABB DEGENERADO



Válido como ABB, pero ineficiente ($O(n)$)

ESTADO OPTIMIZADO: ÁRBOL AVL



Válido como ABB y balanceado como AVL ($O(\log n)$)

Un ABB garantiza orden; un AVL garantiza orden y equilibrio.

La Regla de Equilibrio AVL

Para cada nodo, la diferencia entre la altura del subárbol izquierdo y la altura del subárbol derecho no puede superar 1.

Ejemplo 1 (Válido)



Ejemplo 2 (Inválido)



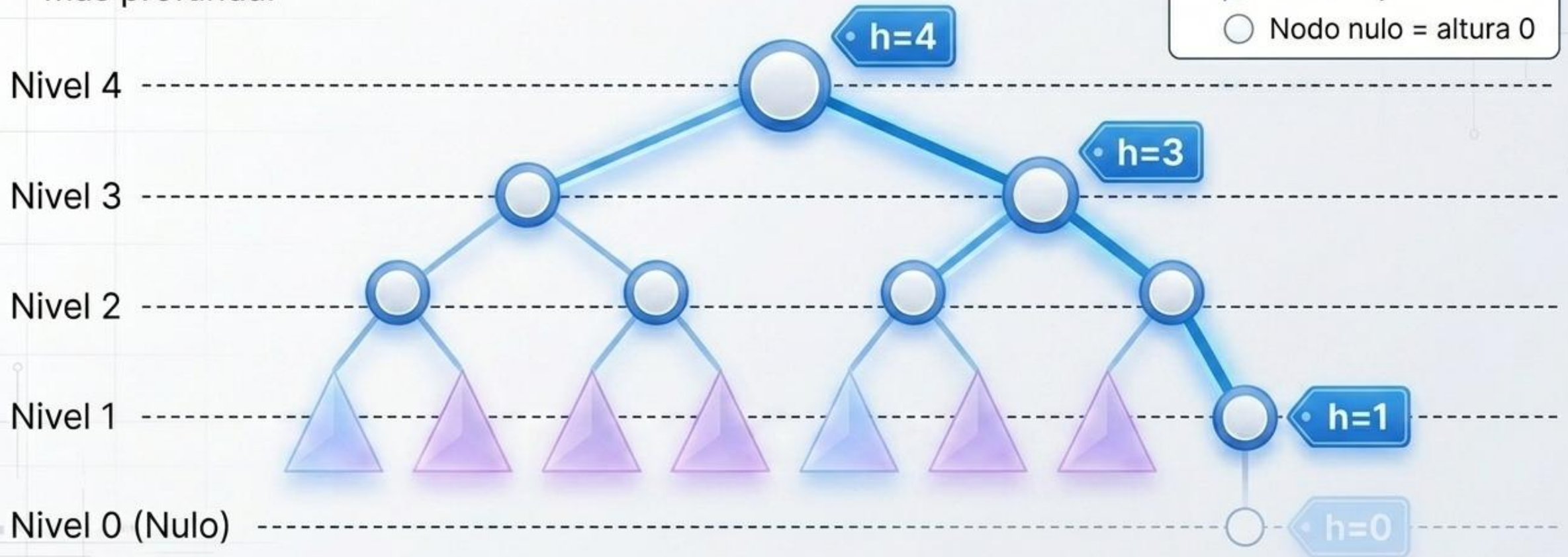
El equilibrio se controla nodo por nodo,
no solo mirando la forma general del árbol.

Midiendo la estructura: Altura de un Nodo

La **altura** es la mayor distancia desde un nodo hasta su hoja más profunda.

Convención de la cátedra:

- 🍃 Nodo hoja = altura 1
- Nodo nulo = altura 0



La **altura** permite estimar el costo máximo de descenso.

Factor de Equilibrio (FE): El Diagnóstico

$$FE = \text{altura}(\text{subárbol derecho}) - \text{altura}(\text{subárbol izquierdo})$$

Estados Válidos



Izquierdo más alto



Lados iguales



Derecho más alto

Estados de Alerta



Requiere corrección



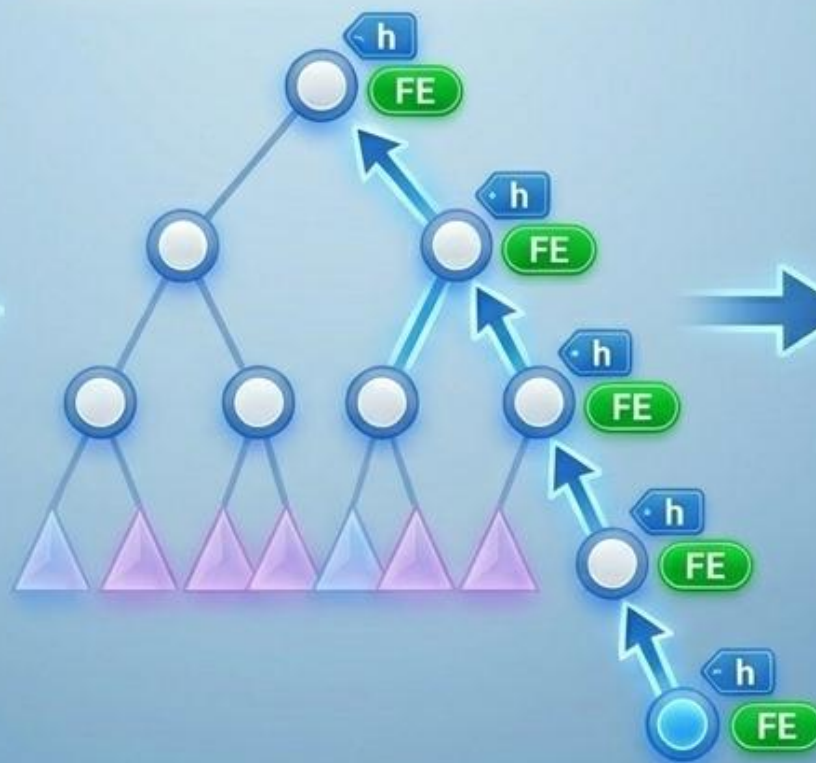
Requiere corrección

Diagnóstico de Balance Post-Insertión

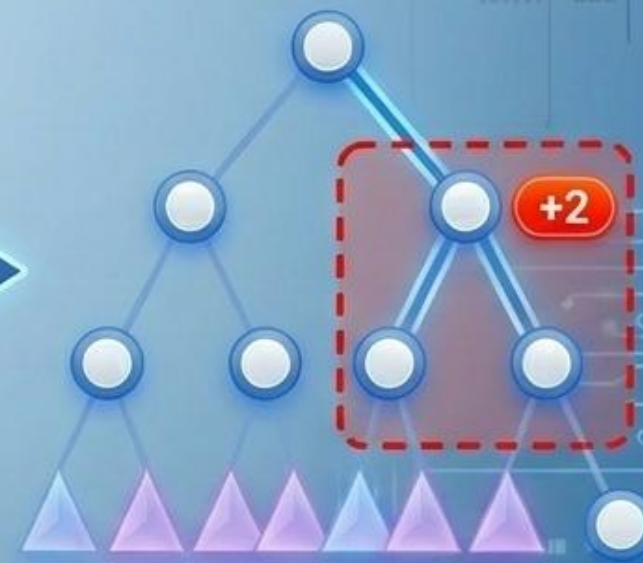
1. Insertar como ABB



2. Revisar camino actualizando Altura y FE



3. Detectar primer nodo fuera de rango



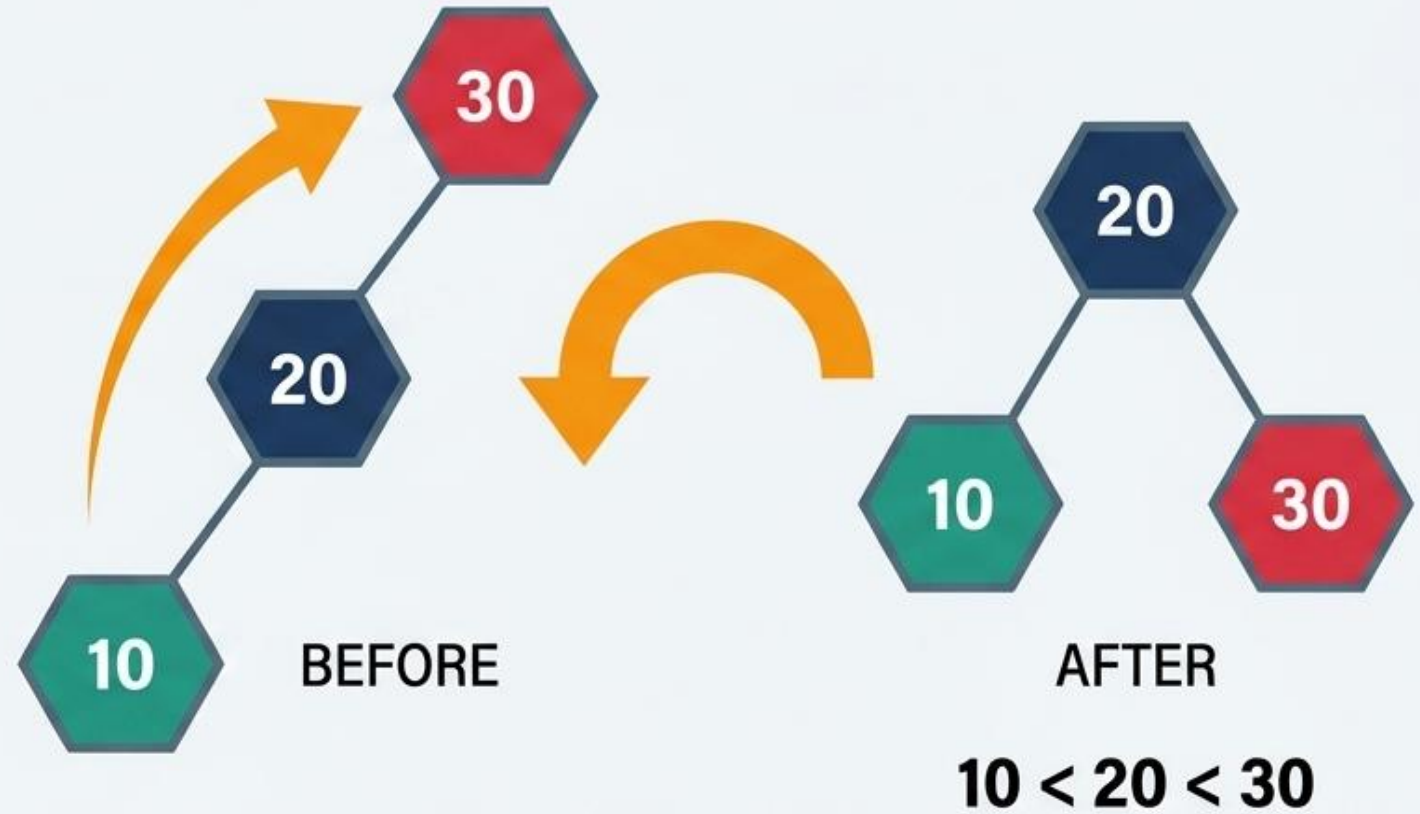
El **factor de equilibrio** indica cuándo intervenir;
las **rotaciones** indican cómo corregir.

Rotaciones AVL: El equilibrio sin perder el orden

¿Qué es? Una reorganización local de nodos que recupera el equilibrio estructural del árbol.

¿Qué logra? Evita el crecimiento lineal (lista) y garantiza operaciones eficientes en tiempo logarítmico, $O(\log n)$.

La Regla de Oro: Cambia las relaciones topológicas (padre-hijo), pero conserva estrictamente el orden lógico de un Árbol Binario de Búsqueda (ABB).

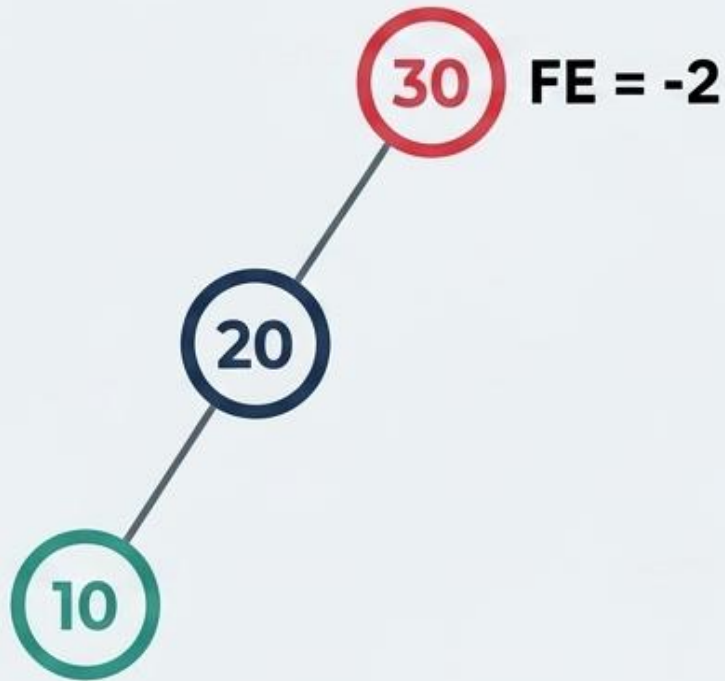


Rotar no es desordenar: es reorganizar para reducir altura manteniendo el orden

Rotación Simple Derecha (Caso Izquierda-Izquierda)

El desbalance ocurre porque el crecimiento **fue lineal hacia la izquierda** (inserción en el subárbol izquierdo del hijo izquierdo).

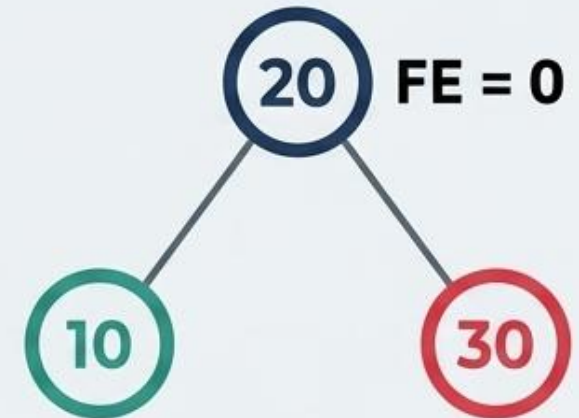
Antes



El Movimiento



Después



Al aplicar una sola rotación en dirección opuesta al crecimiento, la altura del subárbol se reduce instantáneamente, recuperando el balance AVL.

Rotación Simple Izquierda (Caso Derecha-Derecha)

El desbalance ocurre porque el crecimiento fue lineal hacia la derecha (inserción en el subárbol derecho del hijo derecho).

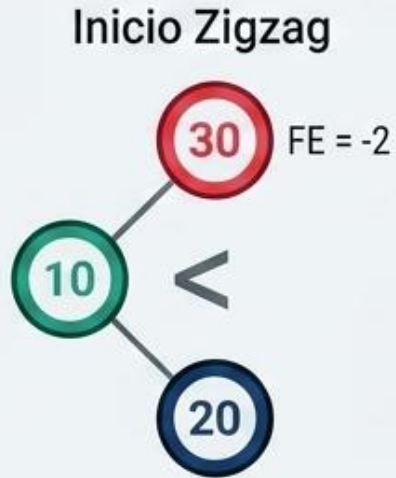


El nodo intermedio asume el control como nueva raíz local.
Todo lo que es menor queda a la izquierda, todo lo mayor a la derecha.

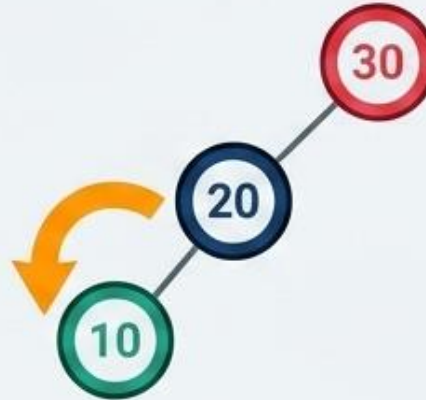
Rotaciones Dobles: Crecimiento en Zigzag

Cuando el crecimiento cambia de dirección, una rotación simple no alcanza. La primera rotación alinea la estructura; la segunda recupera el equilibrio.

Caso
Izquierda-
Derecha
(FE -2)



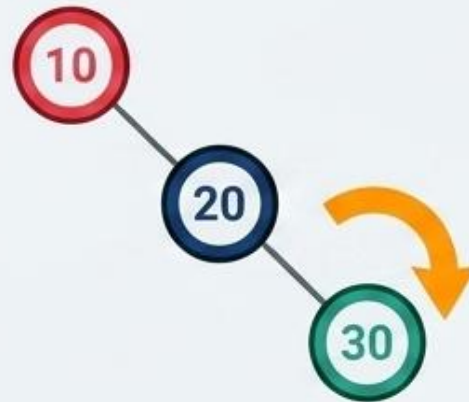
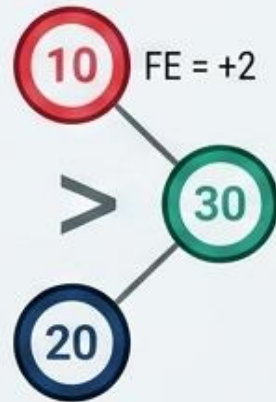
Paso 1: Alinear



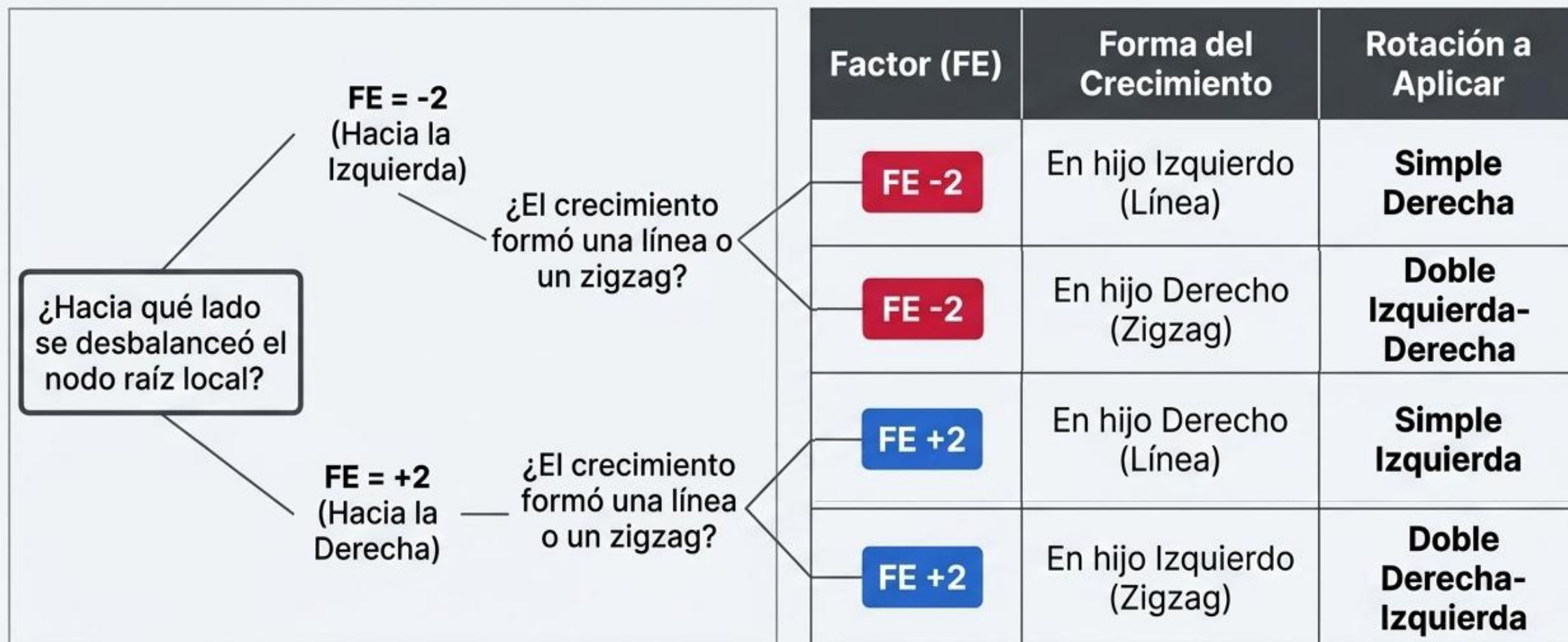
Paso 2: Equilibrar



Caso
Derecha-
Izquierda
(FE +2)

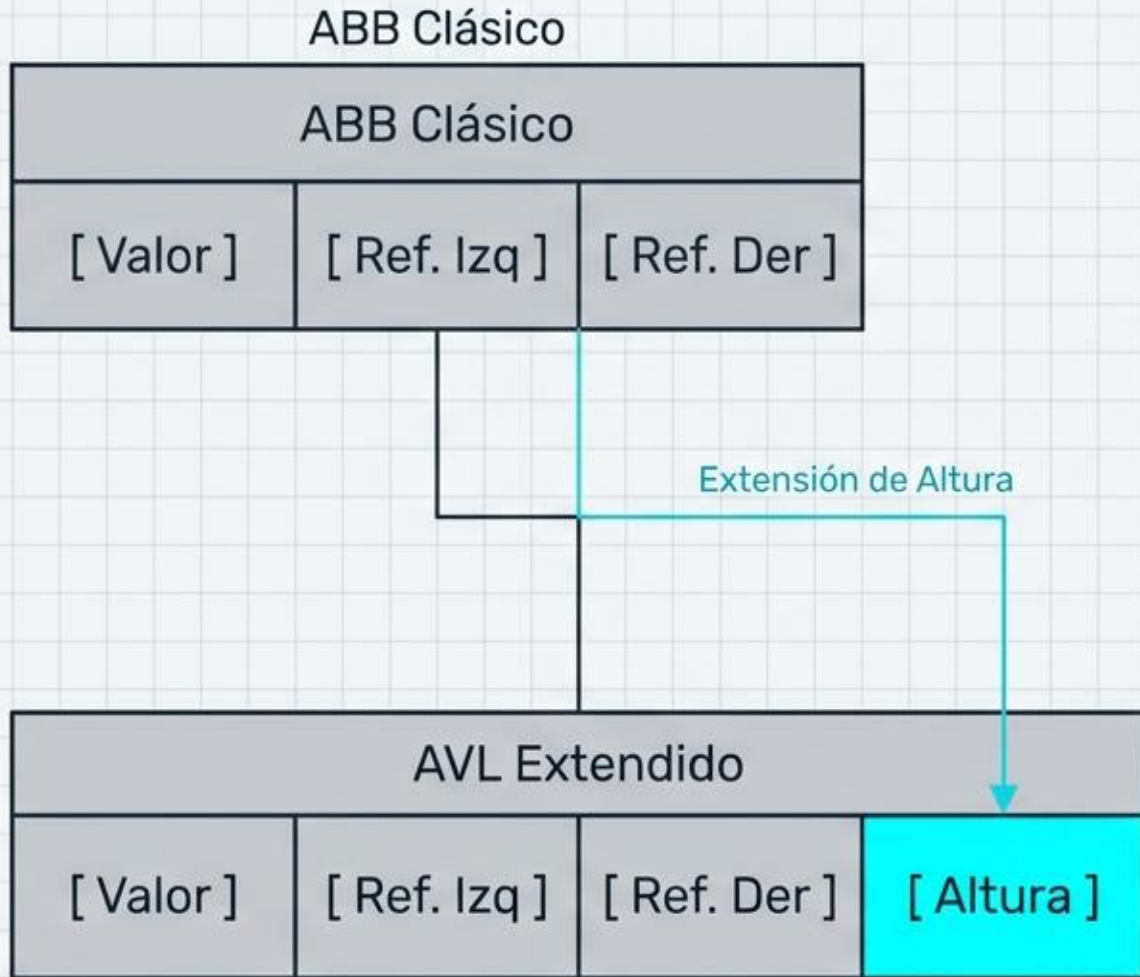


Diagnóstico: ¿Cómo elegir la rotación correcta?



Las rotaciones se eligen por la forma y dirección del desbalance, no por memorización aislada.

Estructura Base: Nodo AVL y Almacenamiento de Altura



```
class NodoAVL {  
    int valor;  
    int altura; // Propiedad clave  
    NodoAVL izquierdo;  
    NodoAVL derecho;  
}
```

¿Por qué guardar la altura?

En un ABB tradicional, calcular la altura de un subárbol requiere recorrer todos sus nodos (costo lineal).

En un AVL, almacenar explícitamente la altura en cada nodo permite calcular el Factor de Equilibrio de forma inmediata (costo constante $O(1)$) consultando a los hijos directamente, sin necesidad de recorrer todo el subárbol cada vez.

Diagnóstico y Mantenimiento: La Lógica del Balance

1. Acceso Seguro

```
int altura(NodoAVL nodo) {  
    if (nodo == null) return 0;  
    return nodo.altura;  
}
```

2. Medición de Desviación

```
int factorEquilibrio(NodoAVL  
nodo) {  
    if (nodo == null) return 0;  
    return altura(nodo.derecho) -  
           altura(nodo.izquierdo);  
}
```



3. Reajuste Interno

```
void actualizarAltura(NodoAVL  
nodo) {  
    nodo.altura = 1 +  
        Math.max(altura(nodo.izq),  
                 altura(nodo.der));  
}
```



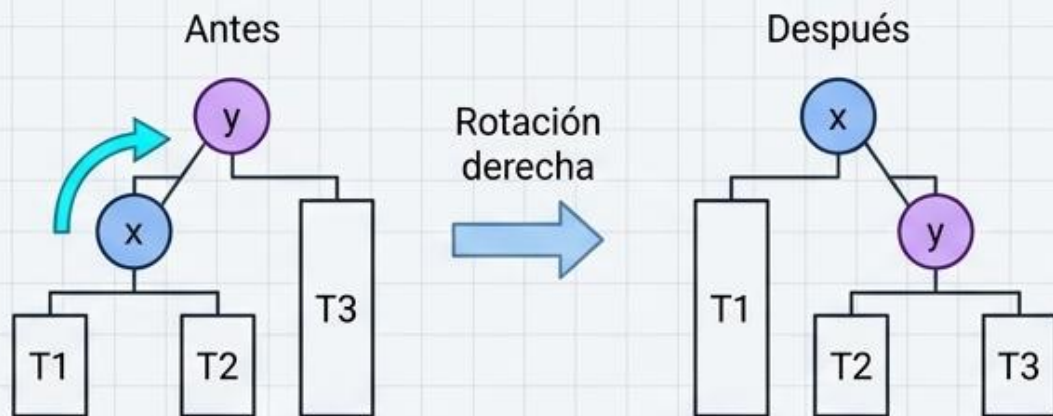
El Peligro de la Inconsistencia Lógica

Un error clásico en la implementación es actualizar las referencias de los hijos (izquierdo, derecho) tras una rotación, pero olvidar invocar `actualizarAltura()`. Si esto ocurre, el árbol puede lucir visualmente correcto, pero su diagnóstico será ciego: tomará decisiones de balanceo incorrectas en el futuro porque la memoria lógica interna ha quedado desactualizada.

Restauración Local: Rotaciones Simples en Código

Rotación Derecha

Corrige desbalance Izquierda-Izquierda



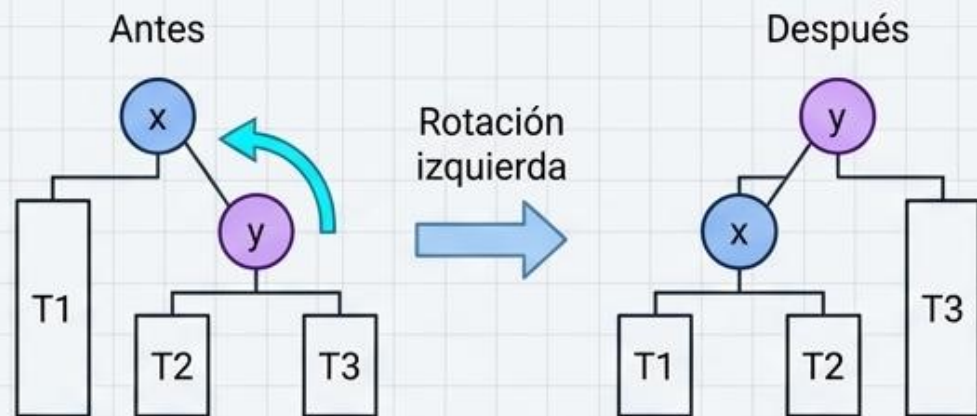
```
x = y.izquierdo;
t2 = x.derecho;

x.derecho = y;
y.izquierdo = t2;

actualizarAltura(y);
actualizarAltura(x);
return x; // Nueva raíz local
```

Rotación Izquierda

Corrige desbalance Derecha-Derecha



```
y = x.derecho;
t2 = y.izquierdo;

y.izquierdo = x;
x.derecho = t2;

actualizarAltura(x);
actualizarAltura(y);
return y; // Nueva raíz local
```



El Secreto de la Reasignación

Observar el último paso: **return x** o **return y**. Las rotaciones alteran la jerarquía física. El nodo que antes era la raíz de este subárbol ya no lo es. Este retorno es vital para que, al deshacer la recursión, el nodo padre actualice su puntero hacia la nueva raíz local, evitando que el subárbol se desconecte.



Inserción AVL: Extensión del ABB con Control de Balance

```
insertar(nodo, valor):  
  // 1. Inserción clásica de ABB  
  si nodo es null, crear y retornar nuevo nodo  
  si valor < nodo.valor, nodo.izq = insertar(nodo.izq, valor)  
  si valor > nodo.valor, nodo.der = insertar(nodo.der, valor)  
  
  // 2 y 3. Diagnóstico al volver de recursión  
  actualizarAltura(nodo)  
  fe = factorEquilibrio(nodo)  
  
  // 4. Corrección local (si corresponde)  
  si fe < -1 aplicar caso izquierdo (Rotación Der / Doble)  
  si fe > 1 aplicar caso derecho (Rotación Izq / Doble)  
  
  retornar nodo // 0 la nueva raíz local si hubo rotación
```



Evolución, no reemplazo:

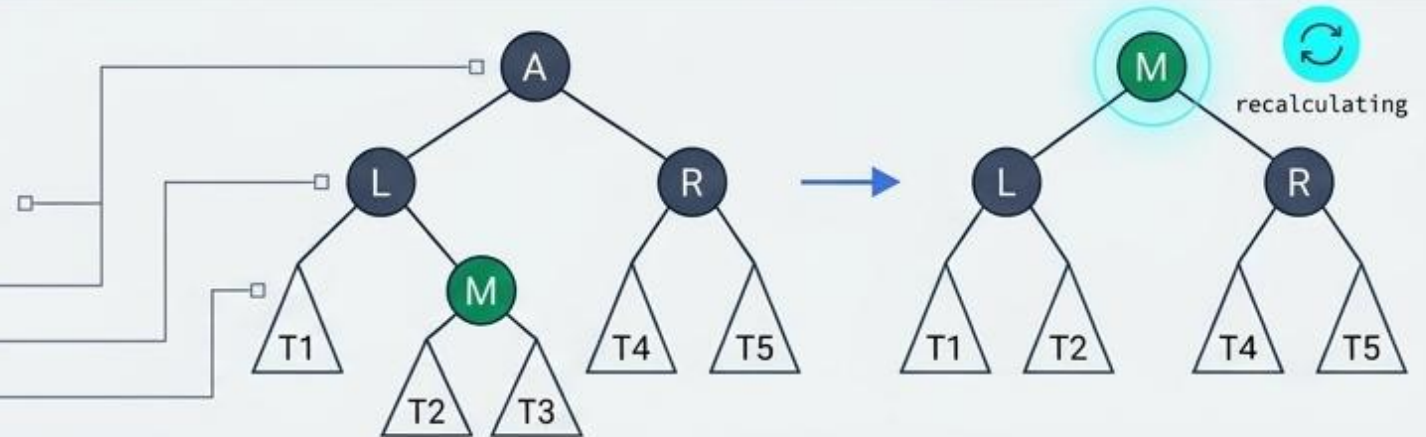
El AVL no altera la regla de orden espacial del ABB. Simplemente añade un control de calidad durante el retorno de la recursión, evaluando localmente si la acción previa rompió la regla de diferencia máxima de **altura = 1**, y reparándolo **on-the-fly**

Eliminación Conceptual y la Promesa Final $O(\log n)$

La Mecánica de Eliminación

- El borrado comparte la arquitectura de la inserción, requiriendo reevaluación hacia la raíz:

1. Eliminar como ABB (Si tiene dos hijos, reemplazar por el mayor del subárbol izquierdo).
2. Actualizar altura en el retorno recursivo.
3. Calcular Factor de Equilibrio.
4. Rotar si corresponde.



Garantía de Costo

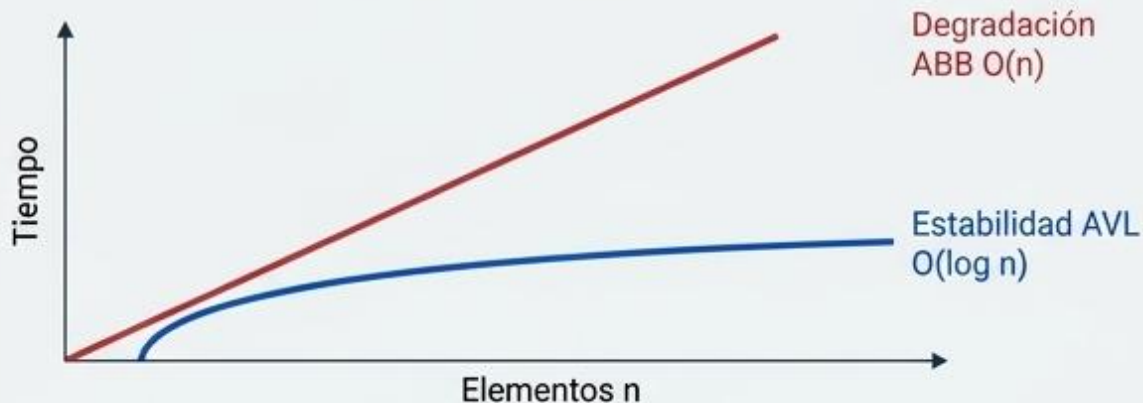


ABB Común: En el peor caso (datos ordenados) se degenera a una lista lineal $O(n)$.
Árbol AVL: El mantenimiento constante asegura que la altura se mantenga controlada. Búsqueda, inserción y eliminación están estrictamente garantizadas en $O(\log n)$.

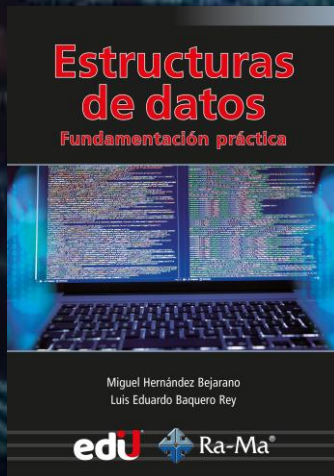
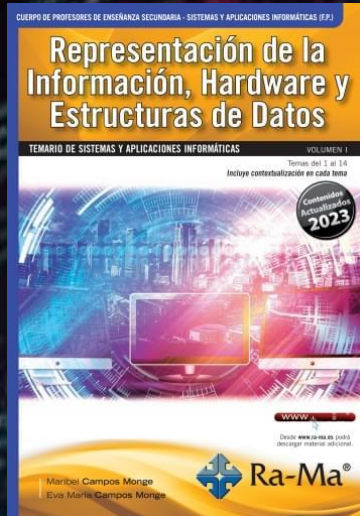
Criterio de Aplicación Arquitectónica

¿Cuándo usar AVL?

Ideal para estructuras en memoria RAM con búsquedas muy frecuentes y tiempos de respuesta que exigen máxima estabilidad operativa.

¿Cuándo usar Árboles B?

Para almacenamiento en disco duro o BD con volúmenes masivos. El factor limitante es la lectura de bloques/páginas físicas, requiriendo árboles muy anchos y de baja altura.



BIBLIOGRAFIA



Cuadrado Estrebou, María Fernanda Trutner, Guillermo Hernán. Revisión: J.Tempio.
Programación II. Apuntes de cátedra.



PROXIMO ENCUENTRO



Clase 11

Arbol B: estructuras para grandes volúmenes

Motivación (accesos a disco). Estructura de Árbol B.
Diferencias con ABB y AVL.